

Deep Research: Avatrada 57 Topic 002

Engineering Report: Zero Gamma (Flip Point) Root Finding

This report provides a detailed technical analysis of the "Zero Gamma (Flip Point) Root Finding" system component. The analysis covers the theoretical underpinnings, a practical implementation strategy in Python, and a critical evaluation of performance, potential failure modes, and underlying assumptions.

1. Technical Deconstruction

The core task is to find the underlying asset price at which the aggregate Gamma Exposure (GEX) of all options in a chain equals zero. This price is referred to as the "Zero Gamma" level or "Flip Point".

1.1. Core Concept: Gamma Exposure (GEX)

Gamma is the second derivative of an option's price with respect to the underlying asset's price. It measures the rate of change of an option's Delta. Gamma Exposure for a single options contract is a measure of the total gamma influence that contract exerts on the market, typically from the perspective of market makers who are net short options.

The formula for GEX for a single strike is:

$$\text{GEX_strike} = \text{Sign} * \text{Gamma} * \text{Open_Interest} * \text{Contract_Size}$$

- **Gamma (\$\Gamma\$):** The gamma of the option at a specific strike. It is a function of the underlying price, strike price, time to expiry, volatility, and risk-free rate.

- **Open Interest (OI):** The number of outstanding contracts for that strike.
- **Contract Size:** For SPX options, this is typically 100.
- **Sign:** This is a crucial convention. Market makers are generally assumed to be net short options to provide liquidity.
 - Shorting a call (which has positive gamma) results in negative gamma exposure for the market maker.
 - Shorting a put (which also has positive gamma) also results in negative gamma exposure.
 - Therefore, the sign is typically **-1** for all options when calculating dealer GEX.

1.2. The Aggregate GEX Function: $f(\text{Price})$

The system requires modeling the *aggregate* GEX as a continuous function of the underlying asset's price, $\$S$. The input data provides a snapshot of gamma values calculated at the *current* market price. To find the Zero Gamma level, we must be able to recalculate gamma for every option in the chain for any *hypothetical* underlying price $\$S$.

This defines our objective function, $f(S)$:

$$f(S) = \sum_{i=1}^N \text{GEX}_i(S) = \sum_{i=1}^N (-1) \cdot \Gamma_i(S) \cdot \text{OI}_i \cdot 100$$

Where: - $\$S$ is the hypothetical underlying price we are solving for. - N is the total number of options contracts in the chain. - $\Gamma_i(S)$ is the recalculated gamma of the i -th option contract at price $\$S$. This requires an options pricing model, typically Black-Scholes-Merton for European options like SPX.

The Black-Scholes formula for Gamma (identical for calls and puts) is:

$$\Gamma(S) = \frac{\phi(d_1)}{S \cdot \sigma \cdot \sqrt{T}}$$

where

$$d_1 = \frac{\ln(S/K) + (r + \frac{\sigma^2}{2})T}{\sigma \sqrt{T}}$$

- $\phi(d_1)$ is the probability density function (PDF) of the standard normal distribution.
- S : Underlying Price
- K : Strike Price
- T : Time to Expiration (in years)
- r : Risk-Free Interest Rate
- σ : Implied Volatility

1.3. The Root-Finding Problem

The "Flip Point" is the price S^* such that the aggregate GEX is zero. The problem is now a classic numerical root-finding problem:

Find S^* such that $f(S^*) = 0$.

This requires a numerical algorithm because the function $f(S)$ is a complex summation with no straightforward analytical inverse.

2. Implementation Strategy

We will use Python with the `pandas`, `numpy`, and `scipy` libraries to implement a robust solution. The strategy involves defining the GEX function and then applying a bracketing root-finder like `scipy.optimize.brentq`.

2.1. Required Libraries

- **pandas**: For managing the options chain data.
- **numpy**: For efficient, vectorized numerical computations.
- **scipy.stats.norm**: For the standard normal PDF required in the gamma calculation.
- **scipy.optimize.brentq**: A robust and efficient root-finding algorithm.

2.2. Step-by-Step Implementation

- 1. Prepare Input Data:** Start with a pandas DataFrame containing, at minimum: `strike`, `type` (call/put), `open_interest`, `iv` (implied volatility), and `tte` (time to expiration in years).
- 2. Define the Aggregate GEX Function:** Create a Python function `calculate_aggregate_gex(S, options_df, r)` that takes a price `S`, the options DataFrame, and the risk-free rate `r` as input.
 - Inside this function, perform a vectorized calculation of d_1 and then Gamma for all options in the `options_df` using the input price `S`.
 - Calculate GEX for each option: $-1 * \text{Gamma} * \text{open_interest} * 100$.
 - Return the sum of all GEX values.
- 3. Bracket the Root:** `brentq` requires a search interval `[a, b]` where `f(a)` and `f(b)` have opposite signs.
 - Start with a reasonable range around the current spot price (e.g., `spot_price ± 20%`).
 - Check if the signs of `f(a)` and `f(b)` are different.
 - If not, systematically expand the search interval until a sign change is detected. This ensures a root exists within the bracket.
- 4. Apply the Root-Finding Algorithm:** Call `scipy.optimize.brentq`, passing the GEX function, the bracket `[a, b]`, and any additional arguments (like the options data).

2.3. Python Code Implementation

```
import pandas as pd
import numpy as np
from scipy.stats import norm
from scipy.optimize import brentq

# --- Core Calculation Functions (Vectorized for Performance) ---

def black_scholes_gamma(S, K, T, r, sigma):
```

```

"""
Calculates Black-Scholes gamma for an array of options.
S: Underlying price(s) - can be a scalar or array
K: Strike price(s) - array
T: Time to expiration (in years) - array
r: Risk-free rate - scalar
sigma: Implied volatility - array
"""

# Handle potential zero division or log of zero
# Add a small epsilon to T and sigma to avoid errors
T = np.maximum(T, 1e-9)
sigma = np.maximum(sigma, 1e-9)

d1 = (np.log(S / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * np.sqrt(T))
pdf_d1 = norm.pdf(d1)

gamma = pdf_d1 / (S * sigma * np.sqrt(T))
return gamma

def calculate_aggregate_gex(S, options_df, r):
    """
    The objective function f(S) for the root finder.
    Calculates total GEX for a given underlying price S.
    """
    # For performance, extract arrays from the dataframe once
    strikes = options_df['strike'].values
    expirations = options_df['tte'].values
    volatilities = options_df['iv'].values
    open_interests = options_df['open_interest'].values

    # Recalculate gamma for all options at the hypothetical price S
    gamma_values = black_scholes_gamma(S, strikes, expirations, r, volatilities)

    # Calculate GEX per option (dealer perspective: net short)
    # GEX is in dollars per 1-point move in the underlying
    gex_per_option = -1 * gamma_values * open_interests * 100

    # Return the sum (total GEX)
    return np.sum(gex_per_option)

```

```

# --- Root Finding Logic ---

def find_zero_gamma_level(options_df, spot_price, r=0.05, search_range=0.20):
    """
    Finds the Zero Gamma (Flip Point) level.

    Args:
        options_df (pd.DataFrame): Must contain 'strike', 'tte', 'iv',
        'open_interest'.
        spot_price (float): The current price of the underlying.
        r (float): Risk-free rate.
        search_range (float): Initial search range (e.g., 0.20 for +/- 20%).

    Returns:
        float or None: The price of the Zero Gamma level, or None if not found.
    """
    # Filter out options with zero open interest to reduce computation
    df = options_df[options_df['open_interest'] > 0].copy()
    if df.empty:
        print("No options with open interest.")
        return None

    # 1. Bracket the root
    a = spot_price * (1 - search_range)
    b = spot_price * (1 + search_range)

    try:
        gex_a = calculate_aggregate_gex(a, df, r)
        gex_b = calculate_aggregate_gex(b, df, r)

        # Expand bracket if root is not found in the initial range
        attempts = 0
        while np.sign(gex_a) == np.sign(gex_b):
            a *= (1 - search_range)
            b *= (1 + search_range)
            gex_a = calculate_aggregate_gex(a, df, r)
            gex_b = calculate_aggregate_gex(b, df, r)
            attempts += 1
            if attempts > 10: # Safety break
                print("Failed to bracket the root. GEX curve may not cross

```

```

zero.")
        return None

    except Exception as e:
        print(f"Error during bracketing: {e}")
        return None

    # 2. Use Brent's method to find the root
    try:
        zero_gamma_price = brentq(
            calculate_aggregate_gex,
            a,
            b,
            args=(df, r),
            xtol=1e-6, # Tolerance
            rtol=1e-6
        )
        return zero_gamma_price
    except ValueError:
        print("Root finding failed. The signs at the bracket endpoints may be
the same.")
        return None

# --- Example Usage ---
if __name__ == '__main__':
    # Create a sample SPX options chain DataFrame
    data = {
        'strike': [4400, 4450, 4500, 4550, 4600, 4650, 4700],
        'open_interest': [5000, 8000, 15000, 12000, 9000, 6000, 4000],
        'iv': [0.22, 0.20, 0.18, 0.16, 0.15, 0.17, 0.19],
        'tte': [10/365.25] * 7 # 10 days to expiration
    }
    sample_df = pd.DataFrame(data)
    current_spx_price = 4530.0

    print(f"Current SPX Price: {current_spx_price}")

    # Calculate GEX at the current price
    current_gex = calculate_aggregate_gex(current_spx_price, sample_df, r=0.05)
    print(f"GEX at current price: ${current_gex:,.0f}")

```

```
# Find the Zero Gamma level
flip_point = find_zero_gamma_level(sample_df, current_spx_price, r=0.05)

if flip_point:
    print(f"\nCalculated Zero Gamma (Flip Point): {flip_point:.2f}")
```

3. Critical Analysis

3.1. Algorithm Selection: Brent's Method vs. Newton-Raphson

- **Brent's Method (`brentq`)**: This is a hybrid algorithm that combines the guaranteed convergence of the bisection method with the speed of the secant method.
 - **Pros**: Extremely robust. As long as a root is bracketed (i.e., `f(a)` and `f(b)` have opposite signs), it is guaranteed to find it. It does not require the derivative of the function.
 - **Cons**: Can be slightly slower than Newton's method if the function is well-behaved and the derivative is easy to compute.
- **Newton-Raphson Method**: This is an iterative method that uses the function's derivative to find successively better approximations of the root.
 - **Pros**: Converges very quickly (quadratically) if the initial guess is close to the root.
 - **Cons**: Requires the derivative of the aggregate GEX function. This derivative is known as aggregate "Speed" (the third derivative of the option price), which adds computational complexity. It can fail to converge if the initial guess is poor, if the derivative is near zero, or if the function has oscillations.

Conclusion: For this application, **Brent's method is the superior choice**. Its robustness and the fact that it does not require calculating aggregate Speed make it far more reliable and simpler to implement than Newton-Raphson.

3.2. Performance Considerations (Per-Minute Execution)

The primary performance bottleneck is the `calculate_aggregate_gex` function, which is called repeatedly by the `brentq` solver. For a full SPX options chain with thousands of strikes, this can be slow.

Optimizations:

1. **Vectorization (Implemented):** The provided code uses `numpy` to perform vectorized calculations across the entire DataFrame at once, avoiding slow Python loops. This is the single most important optimization.
2. **Pre-filtering the Chain:** Options that are very far out-of-the-money (OTM) have negligible gamma. We can filter the DataFrame *before* passing it to the root-finder to significantly reduce the number of calculations per iteration. A filter based on Delta (e.g., `abs(delta) < 0.01`) or a wide strike range is effective.
3. **Intelligent Bracketing:** Instead of starting the bracket search from scratch every minute, use the previously calculated Flip Point as the center for the new search. The Flip Point typically doesn't move dramatically minute-to-minute, so this can drastically narrow the initial search interval `[a, b]`.
4. **JIT Compilation:** For ultimate performance, libraries like `Numba` can be used with the `@jit` decorator on the `black_scholes_gamma` function to compile it to machine code, providing C-like speeds.

3.3. Potential Failure Modes and Edge Cases

- **No Root Exists:** The aggregate GEX curve may not cross zero within a plausible price range (i.e., it's always positive or always negative). The bracketing logic must have a safety break to prevent an infinite loop, after which it should report that no flip point was found.
- **Multiple Roots:** It is possible for the GEX curve to cross zero at multiple points. `brentq` will only find one root within the supplied bracket. The bracketing logic (starting from the spot price and expanding) will naturally find the root closest to the current market. A full analysis would require plotting the GEX curve over a wide price range.

- **Discontinuities in Data:** On expiration days, large parts of the options chain disappear at once, causing the Zero Gamma level to jump significantly. The model will handle this correctly, but the output may be volatile.
- **Bad Input Data:** Missing or zero implied volatility values will cause errors in the Black-Scholes calculation. The input data must be sanitized to handle or remove such rows.

3.4. Model Simplifications and Assumptions

- **Implied Volatility Assumption:** This is the most significant assumption. The model re-prices gamma for a hypothetical price S but uses the *current* implied volatility (iv) for that strike. In reality, the entire volatility skew would shift with a large move in the underlying price. More advanced models use "sticky delta" or "sticky strike" rules to project how IV would change, but using the static IV from the current chain is a standard and acceptable simplification.
- **Constant Rates and Dividends:** The model assumes a constant risk-free rate and no dividend yield (or a constant one). For short-dated options like SPX, this is a minor source of error.
- **Dealer Positioning:** The model assumes dealers are net short *all* options, hence the -1 multiplier. While a strong and widely used convention, the true net position is unknown and varies. The calculated GEX is therefore an *estimate* of market-maker exposure.